

MERN STACK

E-Commerce Web App

Roman Urdu Mein — Zero Se Hero Tak

CHAPTER 7

ROUTING

HOOKS

ROMAN URDU

CHAPTER 7

React Router & Advanced Hooks

Pages banana, navigate karna, global state aur custom hooks — sab kuch!

Is Chapter Mein Aap Seekhenge:

- React Router v6 — install aur BrowserRouter setup
- Routes aur Link — pages banana aur navigate karna
- useParams — URL se dynamic ID lena
- useNavigate — programmatically page change karna
- useContext — global state bina Redux ke
- useReducer — complex state management
- Custom Hooks — reusable logic banana
- Protected Routes — login check karna



ShopEase ke saare pages aur navigation connect karna

■ Chapter 6 Recap

Components, JSX, Props, useState, useEffect, Axios api.js — yeh sab aana chahiye. Chapter 7 mein in components ko pages aur routes se connect karenge!

TABLE OF CONTENTS

1.0 React Router Kya Hai?

- 1.1 SPA vs MPA — farq kya hai?
- 1.2 React Router v6 install
- 1.3 BrowserRouter setup
- 1.4 Router ke main components

2.0 Routes aur Link

- 2.1 Route — page define karna
- 2.2 Link — navigate karna
- 2.3 NavLink — active styling
- 2.4 Navigate — redirect karna

3.0 useParams Hook

- 3.1 URL parameters kya hain?
- 3.2 useParams syntax
- 3.3 Product detail page example

4.0 useNavigate Hook

- 4.1 Programmatic navigation
- 4.2 useNavigate syntax
- 4.3 Login ke baad redirect
- 4.4 History — back/forward

5.0 useContext Hook

- 5.1 Context API kya hai?
- 5.2 Context banana — createContext
- 5.3 Provider — data provide karna
- 5.4 useContext — data consume karna
- 5.5 AuthContext — ShopEase ke liye

6.0 useReducer Hook

- 6.1 useReducer kya hai?
- 6.2 Reducer function
- 6.3 Actions aur dispatch
- 6.4 useReducer vs useState
- 6.5 Cart reducer example

7.0 Custom Hooks

- 7.1 Custom Hook kya hai?
- 7.2 useProducts — data fetch hook
- 7.3 useLocalStorage hook
- 7.4 useAuth hook

8.0 Protected Routes

- 8.1 Protected Route component
- 8.2 Admin Route component
- 8.3 Routes mein apply karna

9.0 ShopEase Complete Routing

- 9.1 App.js — saare routes
- 9.2 Navbar — navigation links
- 9.3 Pages ka connection
- 9.4 Auth flow complete

10.0 Summary & Practice

- 10.1 Chapter recap
- 10.2 Practice exercises
- 10.3 Chapter 8 preview

SECTION 1 — React Router Kya Hai?

1.0 — SPA vs MPA — Pehle Yeh Samjho

Traditional websites mein har page ke liye ek alag HTML file hoti thi — link click karo, poora page reload hota tha. React **SPA (Single Page Application)** banata hai — ek hi HTML file, JavaScript page content badalta hai, koi reload nahi! React Router yeh magic karta hai:

Feature	MPA (Multi Page App)	SPA with React Router ■
Pages	Har page = alag HTML file	Ek HTML — React content swap karta hai
Navigation	Full page reload — slow	Instant — sirf component change
Server requests	Har link pe naya request	Sirf API calls — HTML nahi
Feel	Old school websites	App jaisi — Gmail, Facebook
URL	Hamesha change — server pe jaata	URL change — server pe nahi jaata
Example	Wikipedia, news sites	Gmail, Trello, ShopEase

1.1 — React Router v6 Install

```
# frontend folder mein install karo  
npm install react-router-dom  
  
# Check karo package.json mein:  
# "react-router-dom": "^6.x.x"
```

1.2 — BrowserRouter Setup (src/index.js)

Poori app ko **BrowserRouter** mein wrap karo — yeh routing enable karta hai:

```
// src/index.js  
  
import React    from 'react';  
import ReactDOM from 'react-dom/client';  
import { BrowserRouter } from 'react-router-dom';  
import App      from './App';  
import './index.css';  
  
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```

root.render(
  <React.StrictMode>
    <BrowserRouter>  {/* Yeh wrap zaroori hai! */}
      <App />
    </BrowserRouter>
  </React.StrictMode>
);

// BrowserRouter ke bina useNavigate, Link sab crash kar denge

```

1.3 — Router Ke Main Components

Component/Hook	Import	Kaam
BrowserRouter	react-router-dom	Poori app wrap karo — routing enable
Routes	react-router-dom	Route definitions ka container
Route	react-router-dom	URL se component map karo
Link	react-router-dom	Navigate karo — page reload nahi
NavLink	react-router-dom	Link + active class automatically
Navigate	react-router-dom	Programmatic redirect — JSX mein
Outlet	react-router-dom	Nested routes ka placeholder
useParams	react-router-dom	URL parameters lo — :id
useNavigate	react-router-dom	JS se navigate karo
useLocation	react-router-dom	Current URL info lo
useSearchParams	react-router-dom	Query params ?key=value manage karo

SECTION 2 — Routes aur Link

2.0 — Route — Page Define Karna

Route ek URL path ko ek component se connect karta hai. Jab URL match hota hai toh woh component render hota hai. Sab routes ko **Routes** wrapper mein rakhte hain:

```
// src/App.js

import { Routes, Route } from 'react-router-dom';
import HomeScreen    from './screens/HomeScreen';
import ProductScreen from './screens/ProductScreen';
import CartScreen    from './screens/CartScreen';
import LoginScreen   from './screens/LoginScreen';
import Navbar        from './components/Navbar';

function App() {
  return (
    <>
      <Navbar />    {/* Har page pe dikhega */}

      <main style={{ padding: '20px', maxWidth: '1200px', margin: '0 auto' }}>

        <Routes>

          {/* path="/" = home page */}
          <Route path="/"                element={<HomeScreen />} />

          {/* :id = dynamic parameter – koi bhi ID ho sakti */}
          <Route path="/product/:id"    element={<ProductScreen />} />

          <Route path="/cart"           element={<CartScreen />} />
          <Route path="/login"           element={<LoginScreen />} />
          <Route path="/register"        element={<RegisterScreen />} />
          <Route path="/profile"         element={<ProfileScreen />} />
          <Route path="/orders"         element={<OrdersScreen />} />

          {/* 404 – koi bhi route match na ho */}
          <Route path="*"                element={<NotFoundScreen />} />

        </Routes>

      </main>
    </>
  );
}
```

```

    </main>
  </>
);
}

export default App;

```

2.1 — Link aur NavLink — Navigate Karna

HTML mein `<a href>` se page reload hoti hai. React mein **Link** use karo — page reload nahi hogi:

[illegible]


```

        fontWeight: isActive ? 'bold' : 'normal',
      }}}
    >
      Home
    </NavLink>

    <NavLink to='/cart'
      style={({ isActive }) => ({ color: isActive ? '#F0A500' : '#8B949E' })}
    >
      Cart ■
    </NavLink>
  </nav>
);
}

```

■ YAAD RAHO

Kabhi bhi `<a href>` use mat karo React app mein — page reload ho jaata hai aur poori React app fresh start hoti hai. Hamesha Link ya NavLink use karo!

SECTION 3 — useParams Hook

3.0 — URL Parameters Kya Hain?

Route mein colon (:) se dynamic values define karte hain. Jaise `/product/:id` mein `:id` koi bhi value ho sakti hai — product ka MongoDB `_id`. `useParams` se yeh value component mein milti hai:

```
// Route define karna (App.js mein):  
  
<Route path='/product/:id' element={<ProductScreen />} />  
  
// URL examples:  
  
// /product/64f8a3b2c1d2e3f4a5b6c7d8 -> id = '64f8a3b2c1d2e3f4a5b6c7d8'  
// /product/64abc123def456ghi789jkl0 -> id = '64abc123def456ghi789jkl0'
```

3.1 — ProductScreen — useParams Complete Example

■ src/screens/ProductScreen.jsx

```
import { useState, useEffect } from 'react';  
import { useParams, useNavigate, Link } from 'react-router-dom';  
import { getProductById } from '../services/api';  
  
function ProductScreen() {  
  // URL se :id nikalo  
  
  const { id } = useParams();  
  const navigate = useNavigate();  
  
  const [product, setProduct] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState('');  
  const [quantity, setQuantity] = useState(1);  
  
  useEffect(() => {  
    const fetchProduct = async () => {  
      try {  
        const { data } = await getProductById(id);  
        setProduct(data.data);  
      } catch (err) {
```

```

        setError(err.response?.data?.message || 'Product nahi mila!');
    } finally {
        setLoading(false);
    }
};

fetchProduct();
}, [id]); // id change ho toh dobara fetch karo

const addToCartHandler = () => {
    // Cart mein add karo (Chapter 8 mein Redux se karenge)
    navigate(`/cart?id=${product._id}&qty=${quantity}`);
};

if (loading) return <p style={{color:'#61DAFB'}}>Loading...</p>;
if (error) return (
    <div>
        <p style={{color:'#FF6B6B'}}>{error}</p>
        <Link to="/">Home pe wapas jao</Link>
    </div>
);

return (
    <div style={{ display:'grid', gridTemplateColumns:'1fr 1fr', gap:'40px' }}>
        {/* Left - Image */}
        <img src={product.image} alt={product.name}
            style={{ width:'100%', borderRadius:'12px' }} />
        {/* Right - Details */}
        <div>
            <h1 style={{color:'#E6EDF3'}}>{product.name}</h1>
            <p style={{color:'#56CF8A',fontSize:'24px',fontWeight:'bold'}}>
                Rs. {product.price?.toLocaleString()}
            </p>
            <p style={{color:'#8B949E'}}>{product.description}</p>
            {/* Quantity selector */}
            {product.stock > 0 && (

```

```

        <div style={{display:'flex',gap:'12px',alignItems:'center',margin:'16px 0'}}>
            <button onClick={() => setQuantity(q => Math.max(1, q - 1))}>
-</button>

            <span style={{color:'white'}}>{quantity}</span>
            <button onClick={() => setQuantity(q => Math.min(product.stock, q + 1))}>+</button>
        </div>
    )}

    <button
        onClick={addToCartHandler}
        disabled={product.stock === 0}
        style={{
            background: product.stock > 0 ? '#F0A500' : '#30363D',
            color:      'white',
            padding:    '12px 24px',
            border:     'none',
            borderRadius:'8px',
            cursor:     product.stock > 0 ? 'pointer' : 'not-allowed',
        }}
    >
        {product.stock > 0 ? 'Add to Cart' : 'Out of Stock'}
    </button>
</div>
</div>

    );
}

export default ProductScreen;

```

SECTION 4 — useNavigate Hook

4.0 — Programmatic Navigation

Kabhi kabhi hum JavaScript code se navigate karna chahte hain — button click pe nahi, balki kisi kaam ke baad. Jaise login ke baad home pe, ya cart empty hone pe shop pe. **useNavigate** yeh kaam karta hai:

```
import { useNavigate } from 'react-router-dom';

function LoginScreen() {
  const navigate = useNavigate();

  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const submitHandler = async (e) => {
    e.preventDefault();
    try {
      const { data } = await loginUser(email, password);
      // Token save karo
      localStorage.setItem('token', data.token);
      localStorage.setItem('user', JSON.stringify(data.data));

      // Login ke baad home pe jao
      navigate('/');

      // Ya phir jahan se aaya tha wahan jao:
      // const location = useLocation();
      // navigate(location.state?.from || '/');
    } catch (err) {
      console.error('Login failed:', err);
    }
  };

  return (
    <form onSubmit={submitHandler}>
      <input type='email' value={email} onChange={e => setEmail(e.target.value)} />
    </form>
  );
}
```

```

      <input type='password' value={password} onChange={e => setPassword(
e.target.value)} />

      <button type='submit'>Login</button>

    </form>

  );
}

```

4.1 — navigate() Ke Options

<code>navigate('/')</code>	Home page pe jao — simple navigation.
<code>navigate('/product/'+id)</code>	Dynamic URL pe jao — string concatenation ya template literal.
<code>navigate(-1)</code>	Browser history mein ek step peeche — jaise browser ka Back button.
<code>navigate(1)</code>	History mein ek step aage — Forward button.
<code>navigate('/login', { replace: true })</code>	Navigate karo lekin history mein save mat karo — back pe jaane pe yeh page nahi aayega.
<code>navigate('/login', { state: { from: '/profile' } })</code>	State ke saath navigate karo — destination page useLocation se yeh state read kar sakta hai.

SECTION 5 — useContext Hook

5.0 — Context API Kya Hai?

Socho user login kiya — uski info (naam, token) sirf LoginScreen ke paas hai. Ab Navbar mein naam dikhana hai, Profile mein bhi, Orders mein bhi. Har jagah props pass karna bahut mushkil ho jaata hai — ise **Prop Drilling** kehte hain. **Context API** is masle ka hal hai — data globally available karo:

■ Prop Drilling (Bura)	■ Context API (Acha)
App -> Navbar (user prop) App -> Home -> Product (user prop) App -> Profile (user prop) Har jagah manually pass karo!	AuthContext banao Provider se data provide karo Jis component ko chahiye useContext se lo Koi prop drilling nahi!

5.1 — AuthContext — ShopEase Ke Liye

■ src/context/AuthContext.jsx

```
import { createContext, useContext, useState, useEffect } from 'react';

// Step 1: Context banao
const AuthContext = createContext();

// Step 2: Provider component – data provide karta hai
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(null);

  // App start hone pe localStorage se user load karo
  useEffect(() => {
    const savedUser = localStorage.getItem('user');
    const savedToken = localStorage.getItem('token');
    if (savedUser && savedToken) {
      setUser(JSON.parse(savedUser));
      setToken(savedToken);
    }
  }, []);
```

```

// Login function
const login = (userData, tokenData) => {
  setUser(userData);
  setToken(tokenData);
  localStorage.setItem('user', JSON.stringify(userData));
  localStorage.setItem('token', tokenData);
};

// Logout function
const logout = () => {
  setUser(null);
  setToken(null);
  localStorage.removeItem('user');
  localStorage.removeItem('token');
};

// Context ki value – yeh sab child components ko milega
const value = {
  user,
  token,
  login,
  logout,
  isLoggedIn: !!user,          // Boolean
  isAdmin:   user?.role === 'admin',
};

return (
  <AuthContext.Provider value={value}>
    {children}
  </AuthContext.Provider>
);
}

// Step 3: Custom hook – asaan access ke liye
export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be inside AuthProvider!');
}

```



```
    return context;
  }
```

```
// index.js mein AuthProvider add karo:
```

```
<BrowserRouter>
```

```
  <AuthProvider    {/* Context wrap */}>
```

```
    <App />
```

```
  </AuthProvider>
```

```
</BrowserRouter>
```

```
// Kisi bhi component mein use karo:
```

```
import { useAuth } from '../context/AuthContext';
```

```
function Navbar() {
```

```
  const { user, isLoggedIn, logout } = useAuth();
```

```
  return (
```

```
    <nav>
```

```
      {isLoggedIn
```

```
        ? <><span>{user.name}</span><button onClick={logout}>Logout</button></>
```

```
        : <Link to='/login'>Login</Link>
```

```
      }
```

```
    </nav>
```

```
  );
```

```
}
```

SECTION 6 — useReducer Hook

6.0 — useReducer Kya Hai?

Jab state complex ho jaaye — jaise shopping cart mein add, remove, quantity change, clear — toh bahut saare useState manage karna mushkil hota hai. **useReducer** ek pattern hai jo state changes ko organized rakhta hai. Redux ka chhota bhai samjho!

Feature	useState	useReducer
Best for	Simple, independent values	Complex, related state
Update karna	setter function directly	dispatch(action) se
Logic kahan	Component mein scattered	Reducer function mein centralized
Example	loading, error, name	Cart items, user, products list

6.1 — Cart Reducer — Complete Example

```
■ src/context/CartContext.jsx
```

[illegible]

```

        // Pehle se hai – quantity badhao
        return {
            ...state,
            items: state.items.map(i =>
                i._id === action.payload._id
                ? { ...i, quantity: i.quantity + 1 }
                : i
            ),
        };
    };

    // Naya item add karo

    return { ...state, items: [...state.items, { ...action.payload, quantity: 1 }] };
}

case CART_ACTIONS.REMOVE_ITEM:
    return {
        ...state,
        items: state.items.filter(i => i._id !== action.payload),
    };

case CART_ACTIONS.UPDATE_QTY:
    return {
        ...state,
        items: state.items.map(i =>
            i._id === action.payload.id
            ? { ...i, quantity: action.payload.qty }
            : i
        ),
    };

case CART_ACTIONS.CLEAR_CART:
    return { ...state, items: [] };

default: return state;
}
}

```

[illegible]

TIP

Reducer function hamesha NEW state return karta hai — purani state mutate nahi karta. Spread operator (...state) se poori state copy karo, phir sirf badlne wali cheez update karo. Yeh immutability React ka core principle hai!

SECTION 7 — Custom Hooks

7.0 — Custom Hook Kya Hai?

Socho har screen mein yahi pattern repeat ho raha hai: useState loading, useState error, useEffect fetch. DRY principle! Yeh logic ek **Custom Hook** mein nikalo — ek baar likho, har jagah use karo. Custom Hook sirf ek function hai jo 'use' se shuru hota hai:

7.1 — useFetch — Generic Data Fetch Hook

■ src/hooks/useFetch.js

```
import { useState, useEffect } from 'react';

// Generic fetch hook — koi bhi API call ke liye

function useFetch(apiFn, deps = []) {

  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {

    let cancelled = false; // Cleanup ke liye

    const fetchData = async () => {

      try {

        setLoading(true);

        setError('');

        const { data: result } = await apiFn();

        if (!cancelled) setData(result.data);

      } catch (err) {

        if (!cancelled) setError(err.response?.data?.message || 'Error!');

      } finally {

        if (!cancelled) setLoading(false);

      }

    };

    fetchData();

  }, deps);

}
```

```

    // Cleanup – agar component unmount ho jaaye
    return () => { cancelled = true; };
  }, deps); // eslint-disable-line react-hooks/exhaustive-deps

  return { data, loading, error };
}

export default useFetch;

```

```

// Use karna – kitna clean!

import useFetch from '../hooks/useFetch';
import { getProducts } from '../services/api';

function HomeScreen() {
  const { data: products, loading, error } = useFetch(getProducts);

  if (loading) return <p>Loading...</p>;
  if (error)   return <p>{error}</p>;

  return (
    <div>{products?.map(p => <ProductCard key={p._id} product={p} />)}</div>
  );
}

// Ek line mein products fetch – loading, error sab handle!

```

7.2 — useLocalStorage — Persistent State

■ src/hooks/useLocalStorage.js

```

import { useState } from 'react';

// localStorage + useState ka combo

function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch {
      return initialValue;
    }
  })
}

```

```

});

const setValue = (value) => {
  try {
    setStoredValue(value);
    localStorage.setItem(key, JSON.stringify(value));
  } catch (error) {
    console.error('localStorage error:', error);
  }
};

return [storedValue, setValue];
}

export default useLocalStorage;

// Use karna:

const [theme, setTheme] = useLocalStorage('theme', 'dark');

// Page refresh ke baad bhi 'dark' rahega!

```

7.3 — Custom Hook Ke Rules

Rule	Detail
'use' se shuru	'use' prefix zaroori hai — React tabhi samjhega ke yeh hook hai. useFetch, useAuth, useCart.
Sirf React functions mein	Custom hooks ko sirf components ya doosre hooks mein call karo — regular JS functions mein na
useState/useEffect use	Andar built-in hooks use kar sakte ho — isliye yeh itna powerful hai.
Value return karo	Array ya object return karo — jaise useState [value, setter] return karta hai.

SECTION 8 — Protected Routes

8.0 — Protected Route Kya Hai?

Kuch pages sirf logged in users ke liye hain — Profile, Orders, Checkout. Agar koi directly URL type kare toh use login page pe bhejo. Yeh **Protected Route** karta hai — ek wrapper component:

```
■ src/components/ProtectedRoute.jsx
```

[illegible]


```

        return <Navigate to="/" replace />; // Home pe bhejo – not authorize
      }

      return children;
    }

    export { ProtectedRoute, AdminRoute };

```

8.1 — App.js Mein Apply Karna

```

import { ProtectedRoute, AdminRoute } from '../components/ProtectedRoute';

function App() {
  return (
    <Routes>
      {/* Public pages – koi bhi dekh sakta hai */}
      <Route path="/" element={<HomeScreen />} />
      <Route path="/product/:id" element={<ProductScreen />} />
      <Route path="/login" element={<LoginScreen />} />
      <Route path="/register" element={<RegisterScreen />} />

      {/* Protected pages – login zaroori */}
      <Route path="/profile" element={
        <ProtectedRoute><ProfileScreen /></ProtectedRoute>
      } />
      <Route path="/cart" element={
        <ProtectedRoute><CartScreen /></ProtectedRoute>
      } />
      <Route path="/orders" element={
        <ProtectedRoute><OrdersScreen /></ProtectedRoute>
      } />
      <Route path="/checkout" element={
        <ProtectedRoute><CheckoutScreen /></ProtectedRoute>
      } />

      {/* Admin pages – admin role zaroori */}
      <Route path="/admin/products" element={
        <AdminRoute><AdminProductsScreen /></AdminRoute>

```

```
    } />

    <Route path='/admin/orders' element={
      <AdminRoute><AdminOrdersScreen /></AdminRoute>
    } />

    { /* 404 */ }

    <Route path='*' element={<NotFoundScreen />} />
  </Routes>

);
}
```

■ TIP

Login page pe state={{ from: location.pathname }} bhejte hain. Login hone ke baad LoginScreen yeh state read karta hai aur user ko wahan bhejta hai jahan woh jaana chahta tha — bohat better UX!

SECTION 9 — ShopEase Complete Routing & Navbar

9.0 — index.js — Providers Wrap

■ src/index.js — Final

```
import React    from 'react';
import ReactDOM from 'react-dom/client';

import { BrowserRouter } from 'react-router-dom';
import { AuthProvider }   from '../context/AuthContext';
import { CartProvider }   from '../context/CartContext';
import App from './App';
import './index.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider    /* Auth global state */>
        <CartProvider /* Cart global state */>
          <App />
        </CartProvider>
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

9.1 — Navbar — Complete Component

■ src/components/Navbar.jsx

```
import { Link, NavLink, useNavigate } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import { useCart } from '../../context/CartContext';
```

```

function Navbar() {
  const { user, isLoggedIn, isAdmin, logout } = useAuth();
  const { totalItems } = useCart();
  const navigate = useNavigate();

  const logoutHandler = () => {
    logout();
    navigate('/login');
  };

  const navStyle = {
    display: 'flex',
    justifyContent: 'space-between',
    alignItems: 'center',
    padding: '12px 24px',
    background: '#161B22',
    borderBottom: '1px solid #30363D',
    position: 'sticky',
    top: 0,
    zIndex: 100,
  };

  const linkStyle = (isActive) => ({
    color: isActive ? '#F0A500' : '#8B949E',
    textDecoration: 'none',
    fontWeight: isActive ? 'bold' : 'normal',
    padding: '4px 8px',
  });

  return (
    <nav style={navStyle}>
      {/* Logo */}
      <Link to="/" style={{ color: '#F0A500', fontSize: '22px', fontWeight:
'bold', textDecoration: 'none' }}>
        ■ ShopEase
      </Link>

      {/* Nav Links */}
    </nav>
  );
}

```

```

<div style={{ display:'flex', gap:'16px', alignItems:'center' }}>
  <NavLink to="/" style={{(isActive)} => linkStyle(isActive)}>
Home</NavLink>

  {/* Cart with badge */}
  <NavLink to="/cart" style={{(isActive)} => linkStyle(isActive)}>
    ■ Cart
    {totalItems > 0 && (
      <span style={{
        background:'#FF6B6B', color:'white',
        borderRadius:'50%', fontSize:'11px',
        padding:'2px 6px', marginLeft:'4px',
      }}>
        {totalItems}
      </span>
    )}
  </NavLink>

  {/* Admin link */}
  {isAdmin && (
    <NavLink to="/admin/products" style={{(isActive)} => linkStyle(
isActive)}}>
      Admin
    </NavLink>
  )}

  {/* Auth buttons */}
  {isLoggedIn ? (
    <>
      <NavLink to="/profile" style={{(isActive)} => linkStyle(isAct
ive)}}>
        ■ {user?.name?.split(' ')[0]}
      </NavLink>
      <button onClick={logoutHandler} style={{
        background:'none', border:'1px solid #FF6B6B',
        color:'#FF6B6B', padding:'4px 12px', borderRadius:'6px',
        cursor:'pointer',

```

```

        >>>
        Logout
      </button>

    </>
  ) : (
    <NavLink to='/login' style={({isActive}) => linkStyle(isActive)}>
  >
    Login
  </NavLink>
  )}
</div>
</nav>

);
}

export default Navbar;

```

SECTION 10 — Summary, Practice & Next Steps

10.0 — Chapter 7 Ka Recap

■ React Router v6	SPA routing — page reload nahi, instant navigation
■ BrowserRouter	index.js mein wrap karo — routing enable karta hai
■ Routes + Route	URL path ko component se map karna
■ Link	<a> ki jagah — page reload nahi hoti
■ NavLink	Link + isActive — current page highlight karna
■ useParams	URL se :id ya koi bhi dynamic value nikalna
■ useNavigate	JS se navigate karo — navigate('/'), navigate(-1)
■ useLocation	Current URL, pathname, state info lena
■ Context API	Global state — prop drilling se bachao
■ createContext	Context object banana — data container
■ Provider	Data provide karna — children ko wrap karo
■ useContext	Context se data consume karna — simple!
■ AuthContext	User, token, login(), logout(), isAdmin global
■ useReducer	Complex state — reducer function + dispatch(action)
■ CartContext	Cart items, totalItems, totalPrice, add/remove functions
■ Custom Hooks	'use' se shuru — useState/useEffect wrap karke reuse
■ useFetch	Generic fetch hook — loading, error, data ek call mein
■ ProtectedRoute	Login check — nahi hua toh /login pe Navigate
■ AdminRoute	Login + admin role check — double protection
■ Navbar	useAuth + useCart + NavLink — complete navigation

10.1 — Practice Exercises

■ Easy Level

1. react-router-dom install karo aur BrowserRouter setup karo
2. HomeScreen, LoginScreen, RegisterScreen ke routes banao
3. Navbar mein Link/NavLink se navigation lagao
4. useNavigate se login ke baad home pe redirect karo

■ Medium Level

1. ProductScreen mein useParams se ID lo aur API call karo
2. AuthContext banao — login/logout/isLoggedIn
3. CartContext banao — useReducer se add/remove
4. Navbar mein cart badge dikhao — totalItems se

■ Challenge Level

1. ProtectedRoute aur AdminRoute components banao
2. Profile, Cart, Orders pe ProtectedRoute lagao
3. Admin routes pe AdminRoute lagao
4. useFetch custom hook banao — HomeScreen mein use karo
5. useLocalStorage hook banao — cart data persist karo
6. Login ke baad redirect — state.from use karo

10.2 — Chapter 8 Ka Preview

Redux Toolkit — Professional State Management

- Redux kya hai aur kyun use karte hain
- Redux Toolkit install aur store setup
- Slices — actions aur reducers ek saath
- useSelector — state read karna
- useDispatch — actions dispatch karna
- createAsyncThunk — API calls with Redux
- ShopEase ka poora state — products, cart, auth Redux mein
- Context API se Redux mein migration

Kamaal! Chapter 7 Complete! ■■■

Tumne aaj React Router se ShopEase ke saare pages connect kiye, Context API se global state banaya, useReducer se cart manage kiya, Custom Hooks likhe aur Protected Routes se security add ki. Yeh sab milke ek complete React application banta hai! **Chapter 8 mein Redux Toolkit se professional state management seekhenge!**